Contexto do projeto para as próximas fases

Atualizado em: 29/10/2026 **Repositório:** `digu-dev/Agente-Enem`

Resumo executivo

Projeto acadêmico para construir um agente de estudos para o ENEM usando RAG/GraphRAG. A arquitetura planejada é um monorepo com React/TypeScript, Spring Boot/Java, serviços Go, Terraform/Azure e GitHub Actions. A Fase 0 (walking skeleton) foi entregue e implantada. O corpus ENEM, ingestão, Azure AI Search, grafo Neo4j e funcionalidades pedagógicas ainda são etapas futuras; não trate os prompts como implementação já existente.

Estado confirmado

- PRs #1 a #8 foram mergeados. O PR #9 atualiza os prompts/documentação e está aberto, aguardando checks/revisão.
- A pipeline da `main` passou por lint, testes, Terraform, deploy e smoke test dos três serviços após o merge do walking skeleton.
- O workflow `Destroy dev environment` foi executado em 29/10/2026 e terminou com sucesso. Portanto, os recursos Terraform de `infra/envs/dev` devem estar desligados/removidos. Confirme no portal Azure antes de assumir que não há cobrança residual.
- Para voltar a trabalhar na nuvem: GitHub → Actions → **Walking skeleton pipeline** → Run workflow na `main`. Aguarde apply, build, deploy e smoke test concluírem.
- Para desligar depois: Actions → **Destroy dev environment** → Run workflow na `main` → informar `DESTROY`.
- O state/bootstrap e as identidades OIDC são mantidos pelo workflow de destroy; não são recriados a cada sessão.
- O Cloud Shell é efêmero. Arquivos locais e configurações salvas apenas nele podem desaparecer após desconexão/reinício. O setup está em `scripts/azure-setup.sh`; antes de repetir qualquer etapa de criação, verificar cuidadosamente os recursos e variáveis existentes.
- `apps/ingestion` está planejado, mas não fazia parte do walking skeleton entregue. Não há corpus carregado nem GraphRAG funcional confirmado.

Estrutura e tecnologias atuais

- `apps/web`: React, TypeScript, Vite e Docker.
- `apps/api`: Spring Boot/Java e Docker.
- `apps/retrieval-svc`: Go e Docker.
- `infra/bootstrap`: state remoto Terraform.
- `infra/modules/`, `infra/envs/dev`: infraestrutura modular e ambiente dev.
- `.github/workflows/pipeline.yml`: qualidade, Terraform plan e deploy.
- `.github/workflows/destroy-dev.yml`: teardown manual e confirmado na `main`.
- `scripts/azure-setup.sh`: setup interativo Azure/GitHub, OIDC e permissões.
- `docs/FASE-0-WALKING-SKELETON.md` e

`docs/COMO-CONECTAR-COM-AZURE.md`: documentação existente.

- `Prompts para GitHub Copilot · Agente ENEM com GraphRAG.md`: plano de prompts por fase; a atualização com corpus/custos está no PR #9.

Decisões e cuidados importantes

- GitHub Actions autentica no Azure por OIDC, sem client secret.
- Para o Terraform plan, a identidade de plan recebeu uma role customizada contendo apenas `Microsoft.App/containerApps/listSecrets/action`, pois Reader não bastava para planejar Container Apps existentes.
- A credencial OIDC de apply/deploy usa a branch `main`; workflows de infraestrutura devem rodar na `main` e preservar concorrência `azure-dev-terraform`.
- Recursos de dev podem ser ligados/desligados para limitar custo. O destroy remove recursos de `infra/envs/dev`, não o state remoto nem as identidades.
- Key Vault tem soft delete/purge protection. A recuperação ao recriar deve ser validada na prática; se o apply falhar, não altere proteção nem apague vault sem confirmar impacto e pedir autorização.
- Orçamentos Azure alertam; não bloqueiam cobrança. Verificar Cost Management após workloads e teardown.
- Conta, identidade, criação de recursos Azure, alterações de billing e merge de PR devem ser autorizados explicitamente pelo usuário.
- Manter PDFs fora do Git. O RAG deve processar apenas os arquivos do `manifest.json`; holdout não pode entrar no índice nem no grafo.

Próximas fases sugeridas

Fase 1 — Corpus e ingestão

- Definir manifest versionado com nome, SHA-256, ano, caderno, tipo, fonte, split (`index`/`holdout`) e `corpus_version`.
- Baixar manualmente provas/gabaritos oficiais de Matemática (começar por uma prova) e subir para `enem-raw`; não commitar PDFs.
- Implementar pipeline Go idempotente: extração local primeiro, validação, etiquetagem, exportação e relatório de qualidade.
- Enviar somente dados `index` para `enem-processed` e holdout para `enem-golden`; guardar origem, página e versão do corpus em cada item.
- Usar mocks/cache e limites de tokens. Chamadas reais a LLM devem ser explícitas, ter orçamento e começar com uma execução pequena.

Fase 2 — Retrieval e baseline

- Confirmar no Azure portal quota, região, SKU e preço do Azure AI Search e Azure OpenAI antes de provisionar.

- Implementar index schema e indexar somente texto/metadados/vetores de `enem-processed` com `split=index`; excluir PDFs, holdout e itens gerados não revisados.
- Implementar busca híbrida e contrato OpenAPI; semantic ranker fica desligado por padrão e exige opt-in pago.
- Criar harness reprodutível comparando LLM puro e RAG, registrando acurácia, latência, custo e versões.

Fase 2T — Infraestrutura de IA

Planejar separadamente módulos opcionais para Search, OpenAI e Document Intelligence. SKUs pagos devem exigir opt-in explícito; permitir execução sem OpenAI quando a quota for zero. Antes de apply, mostrar recursos/custos no plan e obter autorização do usuário.

Fase 3 — Grafo Neo4j

Começar local com Neo4j Community/Testcontainers. Carregar somente dados processados não-holdout, com queries parametrizadas, limites de profundidade, paginação e testes de idempotência. AuraDB, se usado para demo, é uma ação manual separada e requer verificação de preço/limites.

Fases 4–5 — API/agente e front-end

Adicionar autenticação, autorização por dono, ferramentas allowlisted, abstenção sem evidência e citações rastreáveis. Testar com LLM mockado. Front-end deve sanitizar conteúdo do modelo, validar contratos e usar login Entra somente quando a configuração de tenant estiver pronta.

Fases 6–8 — CI/CD, avaliação e entrega

Adicionar verificações de contratos, dependências, imagens e manifesto; gate de avaliação pequeno/cacheado em PR e avaliação completa manual com orçamento. Avaliar segurança, custo, latência e qualidade. Finalizar README, ADRs, runbooks de deploy/rollback/teardown e roteiro para execução local.

Forma de trabalho e próximo início

- Trabalhar uma fase por vez; apresentar plano antes de editar código.
- Consultar documentação atual para APIs, versões, SDKs, cotas e preços.
- Escrever testes junto com a implementação e executar os comandos relevantes.
- Fazer mudanças pequenas via branch/PR; não mergear nem aplicar recursos Azure sem autorização.
- Antes da próxima sessão de nuvem, confirmar no Azure que o destroy terminou sem recursos residuais. Ligar pela pipeline manual da `main` apenas quando necessário.
- Começar pela Fase 1 somente depois de aprovar formato do manifest, corpus inicial e orçamento para qualquer chamada de IA.